

Covert Channels: Timing Lab

Hiding Messages in Delays (Rust)

2026-09-30

Table of contents

1	Setting Up the Server	2
1.1	A Basic Chat Server	2
1.2	Sending an Overt Message One Character at a Time	4
1.3	Encoding the Covert Message	6
2	Setting Up the Client	7
2.1	A Basic Chat Client	7
2.2	Receiving One Character at a Time	8
2.3	Measuring the Delays	9
2.4	From Times to Characters	10

In the last lecture, you covered different ways that someone might set up and exploit a covert channel. In the last lab, you set up a storage covert channel using the file permissions of an FTP server.

In this lab, you will build a **timing covert channel**. You will set up a server that transmits an ordinary text message one character at a time, but encodes a *secret* message in the pauses between each successive character.

As an example: a delay of 25 ms between characters could represent a 0, while a delay of 100 ms represents a 1. The overt message is perfectly readable by anyone; only a receiver who knows the encoding scheme can recover the hidden message.

In order to transfer a longer covert message, you will need a longer overt message — each overt character carries exactly one covert bit. Using the values above, transmitting a single covert character (7 or 8 bits) requires transmitting at least 8 overt characters. This is not unreasonable: covert channels are typically **low bandwidth**, trading efficiency for secrecy.

To signal the end of the covert message, the sender appends the string "EOF" (encoded in binary like any other text) to the covert bitstream. The receiver stops decoding once it detects this sentinel.

Note

This lab uses **Rust**. You only need the standard toolchain — no external crates. Create a project with `cargo new covert_server` (and another for the client), then edit `src/main.rs`. Build and run with `cargo run`.

1 Setting Up the Server

1.1 A Basic Chat Server

Let's start with a basic TCP server using Rust's standard `std::net` module. This first version acts as an **echo server** — it accepts connections, reads any message the client sends, and echoes it back. This gives you a working baseline to test before adding covert channel logic.

```
/*
 * std::io gives us buffered reading and the Write trait, std::net gives
 * us the TCP types, and std::thread lets us handle each client on its
 * own thread.
 */
use std::io::{BufRead, BufReader, Write};
use std::net::{TcpListener, TcpStream};
use std::thread;

// Constants make the server easy to tweak without hunting through code.
const ADDRESS: &str = "0.0.0.0"; // all IPv4 interfaces.
                                     // Use "127.0.0.1" to restrict to local connections only.
const PORT: &str = "1337";       // choose an unprivileged port (> 1024) not in common use

fn main() {
    // Start listening for TCP connections on the specified address and port.
    let listener = match TcpListener::bind(format!("{ADDRESS}:{PORT}")) {
        Ok(l) => l,
        // If the server can't start (e.g. port already in use), exit with an error.
        Err(e) => {
            eprintln!("Error starting server: {e}");
            std::process::exit(1);
        }
    };

    println!("Server listening on {ADDRESS}:{PORT}");

    // Accept and handle connections indefinitely. `incoming()` yields one
    // result per inbound connection.
    for stream in listener.incoming() {
        match stream {
```

```

    Ok(conn) => {
        println!("New client connected!");

        // Handle each connection on its own thread so the server
        // can serve multiple clients concurrently. `move` hands
        // ownership of the connection to the thread.
        thread::spawn(move || handle_connection(conn));
    }
    Err(e) => {
        // A bad incoming connection shouldn't crash the server -
        // log it and move on to the next one.
        eprintln!("Error accepting connection: {e}");
        continue;
    }
}

}

}

// handle_connection reads messages from the client and echoes them back.
fn handle_connection(stream: TcpStream) {
    // A TcpStream is both readable and writable, but wrapping it in a
    // BufReader takes ownership, so we clone the handle first: one copy
    // to read through, one to write through. Both point at the same socket.
    let mut writer = stream.try_clone().expect("failed to clone stream");
    let mut reader = BufReader::new(stream);
    let mut line = String::new();

    loop {
        line.clear();
        // read_line reads until it finds the delimiter '\n'.
        match reader.read_line(&mut line) {
            // 0 bytes read means the client has disconnected.
            Ok(0) => {
                println!("Client disconnected.");
                return;
            }
            Ok(_) => {
                print!("Message received: {line}");

                // Echo the message back to the client.
                if writer.write_all(format!("You said: {line}").as_bytes()).is_err() {
                    return;
                }
            }
            Err(e) => {
                eprintln!("Error reading from client: {e}");
                return;
            }
        }
    }
}

```

```

    }
  }
}

```

💡 Tip

This is a great place to pause and test your server before adding timing logic. You don't need a custom client yet — use one of these built-in terminal tools: **telnet** (plain text, unencrypted — do not use for sensitive data):

```
telnet localhost 1337
```

netcat (general-purpose network utility):

```
nc localhost 1337
```

Press **Ctrl+C** or type **exit** to disconnect. You can also share your server's IP address with a classmate and have them connect from their machine:

```
telnet ip.ad.re.ss 1337  # or
nc    ip.ad.re.ss 1337
```

1.2 Sending an Overt Message One Character at a Time

The echo server above is just a starting point. To build our covert channel we need to make two key changes:

1. **Stop reading from the client** — our server only needs to *send*, not receive.
2. **Send the overt message one character at a time**, with a deliberate delay after each character. That delay is where the covert message lives.

Bring the sleep and duration types into scope:

```
use std::thread::sleep;
use std::time::Duration;
```

Add timing constants alongside your existing ones:

```
const ADDRESS: &str = "0.0.0.0";
const PORT: &str = "1337";
const ZERO: u64 = 25;    // milliseconds - delay to signal a 0 bit
const ONE: u64 = 100;    // milliseconds - delay to signal a 1 bit
```

The ratio between **ZERO** and **ONE** (here 4×) should be large enough that clock jitter on the receiving side does not cause a misread. A good rule of thumb: the receiver classifies any delay above the midpoint $(\text{ZERO} + \text{ONE}) / 2 = 62.5 \text{ ms}$ as a 1.

! Important

Disable Nagle's algorithm. By default, TCP batches together small outgoing writes (Nagle's algorithm) to cut down on packet overhead. That batching would smear our carefully timed per-character delays together and destroy the covert signal. Go happens to disable Nagle automatically on TCP connections; **Rust does not**, so we call `stream.set_nodelay(true)` ourselves before sending. This pushes each character out in its own packet, the moment we write it.

Now replace `handle_connection` with a new function `send_message` that sends the overt message character by character:

```
fn send_message(message: &str, mut stream: TcpStream) {
    // Send each character immediately instead of letting TCP buffer them.
    let _ = stream.set_nodelay(true);

    for (i, ch) in message.chars().enumerate() {
        if stream.write_all(ch.to_string().as_bytes()).is_err() {
            eprintln!("Error writing to client");
            return;
        }

        /*
         * For now, alternate between one and zero delays based on
         * position - just to verify the timing mechanism works before
         * we wire in the actual covert bitstream.
         */
        if i % 2 == 0 {
            sleep(Duration::from_millis(ONE));
        } else {
            sleep(Duration::from_millis(ZERO));
        }
    }

    println!("Overt message sent. Closing connection.");
    // The stream is dropped here, which closes the connection.
}
```

Update `main` to use the new function:

```
// Inside the `for stream in listener.incoming()` loop, in the Ok(conn) arm:
let message = String::from("This is a perfectly ordinary message that will be sent one character at a time");

// thread::spawn(move || handle_connection(conn)); <- replace this
thread::spawn(move || send_message(&message, conn)); // <- with this
```

1.3 Encoding the Covert Message

We now have a server that sends a message with timing delays. The final step is to replace the alternating dummy delays with delays driven by the actual covert bitstream.

The helper function `get_covert` converts a string to its binary representation — one 8-bit sequence per character, concatenated into a single string. Note that the "EOF" sentinel is appended to the covert message so the receiver knows when to stop decoding. No extra imports are needed; `String` and `format!` are in the standard prelude.

```
fn get_covert() -> String {
    let covert = String::from("secret") + "EOF";
    let mut result = String::new();

    for b in covert.bytes() {
        // {:08b} formats the byte as an 8-bit binary string, left-padded
        // with zeros. For standard ASCII text, iterating the bytes is
        // exactly the character's ASCII value in binary.
        result.push_str(&format!("{b:08b}"));
    }
    result
}
```

As an example: if `covert == "ABC"`, then `get_covert()` returns `"010000010100001001000011"`.

Now update `send_message` to look up the covert bit for each character position and choose the appropriate delay:

```
fn send_message(message: &str, mut stream: TcpStream) {
    let _ = stream.set_nodelay(true);

    let covert = get_covert();
    let covert = covert.as_bytes(); // index into the '0'/'1' characters as bytes

    for (i, ch) in message.chars().enumerate() {
        if stream.write_all(ch.to_string().as_bytes()).is_err() {
            eprintln!("Error writing to client");
            return;
        }

        // Use modulo so the covert message repeats if the overt
        // message is longer than the covert bitstream.
        if covert[i % covert.len()] == b'1' {
            sleep(Duration::from_millis(ONE));
        } else {
            sleep(Duration::from_millis(ZERO));
        }
    }
}
```

```
println!("Overt message sent. Closing connection.");
}
```

💡 Tip

Make sure your overt message is **at least as long as** your covert bitstream. Each covert character requires 8 overt characters. The word "secret" plus the "EOF" sentinel is 9 characters $\times 8$ bits = 72 bits, so the overt message must be at least 72 characters long.

2 Setting Up the Client

💡 Tip

Timing covert channels depend on accurate sleep and clock measurements. These have historically been less reliable on Windows and inside virtual machines. This is our first attempt using Rust — so you will be our guinea pigs. Rust's `Instant` is a monotonic, high-resolution clock and `thread::sleep` is fine for our millisecond-scale delays, but the OS scheduler and TCP buffering still add jitter. If your measurements are inconsistent, try widening the gap between ZERO and ONE, confirm `set_nodelay(true)` is enabled on the server, or run on a native Linux machine.

2.1 A Basic Chat Client

Let's start with a basic client that can connect to the echo server from Section 1.1, send messages, and display the server's responses. This gives you a working foundation before adding timing measurement logic.

```
use std::io::{self, BufRead, BufReader, Write};
use std::net::TcpStream;

fn main() {
    let stream = match TcpStream::connect("localhost:1337") {
        Ok(s) => s,
        Err(e) => {
            eprintln!("Error connecting: {e}");
            std::process::exit(1);
        }
    };

    println!("Connected to server. Type a message and press Enter:");

    // One handle to write through, one (buffered) to read the reply through.
    let mut writer = stream.try_clone().expect("failed to clone stream");
```

```

let mut reader = BufReader::new(stream);

// Read lines from stdin and send them to the server.
let stdin = io::stdin();
for line in stdin.lock().lines() {
    let text = match line {
        Ok(t) => t,
        Err(_) => break,
    };

    if writeln!(writer, "{text}").is_err() {
        break;
    }

    // Read the server's response (terminated by '\n').
    let mut response = String::new();
    match reader.read_line(&mut response) {
        Ok(0) => break, // server closed the connection
        Ok(_) => print!("Server replied: {response}"),
        Err(e) => {
            eprintln!("Error reading from server: {e}");
            break;
        }
    }
}
}
}

```

2.2 Receiving One Character at a Time

The client above is designed for the echo server — it reads whole lines and sends messages. For our covert channel receiver, we need different behaviour:

- The client should **only receive** — it sends nothing to the server.
- It should **display each character as soon as it arrives**, rather than buffering until a newline or connection close.

Our overt message is ASCII, so we read **one byte at a time** and print it immediately. Bring the `Read` trait into scope for the single-byte `read`:

```

use std::io::Read;

// ...

let mut reader = BufReader::new(stream);
let mut byte = [0u8; 1];

loop {

```



```

match reader.read(&mut byte) {
    // read() returning Ok(0) is the true end-of-stream: the server
    // closed the connection. (This is distinct from our "EOF" sentinel,
    // which arrives as ordinary bytes inside the timing pattern.)
    Ok(0) => {
        println!("\nServer closed connection.");
        break;
    }
    Ok(_) => {
        // Print the character immediately as it arrives, and flush so
        // it shows up right away rather than sitting in stdout's buffer.
        print!("{}", byte[0] as char);
        io::stdout().flush().ok();
    }
    Err(e) => {
        eprintln!("\nError reading from server: {e}");
        break;
    }
}
}

```

2.3 Measuring the Delays

We now need to measure the elapsed time between receiving successive characters and classify each gap as a 0 or 1. Use `Instant::now()` before the read and `.elapsed()` after it to get the duration:

```

use std::time::{Duration, Instant};

// ...

// Collect all measured durations for inspection.
let mut times: Vec<Duration> = Vec::new();

// ...

// Inside the receive loop, wrap the read:
let start = Instant::now();
let n = reader.read(&mut byte);
let elapsed = start.elapsed();

times.push(elapsed);

```

After the loop, print the collected durations to inspect them:

```
println!("{times:?}");
```

You should see two clusters of values — one near ZERO milliseconds and one near ONE milliseconds. This visual inspection step is useful for choosing a robust threshold before you write the classification logic.

2.4 From Times to Characters

Once you can observe two distinct timing clusters, converting them to bits is straightforward: any duration above the midpoint threshold is a 1, anything below is a 0.

```
let threshold = Duration::from_millis((ZERO + ONE) / 2);
// threshold = 62 ms with our default constants

let mut covert_bits = String::new();
// For each measured `elapsed`:
if elapsed >= threshold {
    covert_bits.push('1');
} else {
    covert_bits.push('0');
}
```

Once you have accumulated enough bits, convert them back to characters. Process 8 bits at a time. A helper function in Rust might look like this:

```
fn bits_to_string(bits: &str) -> String {
    let mut result = String::new();
    let mut i = 0;

    // bits is a string of '0'/'1' (all ASCII), so byte offsets and char
    // offsets line up and slicing is safe.
    while i + 8 <= bits.len() {
        // Parse one byte (8 bits) as a base-2 integer.
        match u8::from_str_radix(&bits[i..i + 8], 2) {
            Ok(n) => result.push(n as char),
            Err(_) => result.push('?'), // bit error - character undecodable
        }
        i += 8;
    }
    result
}
```

Keep decoding until the recovered string ends with "EOF", then strip the sentinel to reveal the covert message.

i Note

The entire **client** section of this lab has been intentionally vague — all the pieces are here but they are not assembled into a complete, working program. Code snippets are deliberately sparse. It is up to you to design a well-structured Rust program that:

1. Connects to the server
2. Receives the overt message one character at a time and prints it
3. Measures the inter-character delay for each received character
4. Classifies each delay as 0 or 1 using the midpoint threshold
5. Decodes the accumulated bit string into ASCII characters, 8 bits at a time
6. Stops when the "EOF" sentinel is detected
7. Prints the recovered covert message

Good luck.